# Smart Home Alarm Control System: Distributed Cluster Design and Implementation Report

Eduard Toni Alexandru

October 24, 2026

**Abstract**

This report describes the analysis, design and implementation of a smart home alarm control system built with the Apache Pekko framework and deployed on a Pekko Cluster. Sensors, keypads, sirens and the alarm control unit are actors that may run on different cluster nodes and interact only through messages. The control unit runs as a Cluster Singleton, so exactly one instance exists in the whole cluster; every new instance, whether created after a crash, a restart or the loss of its node, starts in a safe recovery mode and requires the correct PIN before returning to the normal state machine.

## 1 Problem Analysis

The smart home alarm domain is inherently concurrent, distributed and reactive. From a concurrent and distributed systems point of view, the main aspects are the following.

- **Autonomous Concurrent Entities:** Sensors, keypads, sirens and the control unit operate independently. Several sensors can detect events at the same time, and a user can type on the keypad while sensor events are in flight.
- **Asynchronous Communication:** The components cannot share memory or rely on blocking calls. Message passing gives loose coupling and avoids race conditions.
- **State-Dependent Message Handling:** The same message has different effects depending on the current state: for example, a sensor event is only logged when the system is disarmed, but starts the entry delay when it is armed.
- **Temporal Constraints:** Exit and entry delays require non-blocking timeouts that trigger state transitions autonomously.

- **Distribution and Location Transparency:** Sensors, keypads and the control unit may run on different nodes. A sensor must be able to reach the control unit without knowing on which node it runs, and every message that crosses nodes must be serialized.
- **A Single Control Entity:** The alarm state must be decided in one place. If two control units were active at the same time, sensors and keypads could talk to different ones, each with its own, inconsistent state.
- **Partial Failures:** A node can crash while the others keep running. If the control unit is lost, a new one must be created, but its previous state (armed or not, active zones, pending timers) is lost with it. The new instance must not guess: assuming *disarmed* would ignore a real intrusion, and assuming *armed* could sound a false alarm.

## 1.1 Functional Requirements

- **System States:**
  - *Disarmed:* sensors do not trigger alarms; normal activity is allowed.
  - *Exit Delay:* entered after arming with the correct PIN. Sensors are still inactive, so that the user can leave. After a fixed timeout the system becomes Armed.
  - *Armed:* sensors are active; an intrusion in an active zone starts the Entry Delay.
  - *Entry Delay:* a countdown starts. The correct PIN before the timeout leads to Disarmed; otherwise the system goes to Alarm.
  - *Alarm:* the siren is activated; only the correct PIN stops it and leads to Disarmed.
- **User Interaction:** the keypad is used to arm, to disarm and to stop the alarm. The PIN is validated before any state transition.
- **Sensors:** each sensor has a `sensorId`; motion sensors and door/window sensors are supported. Sensor events are sent to the control unit. In Disarmed and Exit Delay they are only logged, in Armed they start the Entry Delay, in Entry Delay and Alarm they have no additional effect.
- **Timing Constraints:** configurable exit delay (e.g. 20–60 s) and entry delay (e.g. 10–30 s); timeouts trigger the transitions automatically.

- **Zones:** every sensor belongs to a zone. The user can arm all zones (full arming) or only some of them (partial arming, e.g. night mode with the upper floor inactive); sensors in inactive zones are ignored.
- **Cluster Architecture:** the system runs on at least three cluster nodes; sensors, keypads and the control unit may run on different nodes; communication happens only through actor messages.
- **Failure and Recovery:** when the control unit is restarted or recreated it enters a safe recovery mode. It must assume neither Disarmed nor Armed, sensor events are only logged, and a correct PIN is required to return to Disarmed.

## 2 Design Strategy

The system adopts the Actor Model through the classic Apache Pekko API.

- **Actor-Based Isolation:** the control unit, every keypad, every sensor and every siren is an actor. Each actor processes its messages one at a time and owns its state, so no locks are needed.
- **State Machine with `become()`:** the control unit swaps its message handlers with `getContext().become(...)`. Each state (`recoveryState`, `disarmedState`, `exitDelayState`, `armedState`, `entryDelayState`, `alarmState`) defines exactly which messages it reacts to (Figure 1). Messages that a state does not handle are logged and ignored.
- **Non-Blocking Timers:** exit and entry delays use the single-shot timers of `AbstractActorWithTimers`. Both delays use the same timer key, so starting or cancelling one never leaves a stale timeout behind.
- **Cluster Singleton for the Control Unit:** the control unit is started through a `ClusterSingletonManager` on every node with the `control` role. The managers guarantee that exactly one instance runs, on the oldest `control` node. If that node leaves or is removed after a crash, the instance is created again on another `control` node.
- **Location Transparency with a Proxy:** every node creates a `ClusterSingletonProxy`, a local actor that forwards messages to the control unit wherever it runs. Sensors, keypads and sirens only know this local proxy. While the singleton is being created or moved, the proxy buffers the messages.
- **Serialization:** every message that can cross nodes implements the marker interface `CborSerializable`, which `application.conf` binds

to Pekko's Jackson CBOR serializer. Messages are immutable Java records; sets of zones are copied with `Set.copyOf` when a message is created.

- **Safe Recovery Mode by Construction:** a new instance of an actor always starts with empty fields and its initial behavior. For this reason `createReceive()` returns `recoveryState()`, not `disarmedState()`. Every way of obtaining a new instance (first start, restart by the supervisor after an exception, re-creation on another node) leads to the recovery mode, without any special case in the code.
- **Self-Registering Sirens:** a siren sends `RegisterSiren` to the control unit when it starts and then every 3 seconds. A new control unit, which has lost its list of sirens, finds them again within a few seconds. The control unit watches each siren, so it removes a siren whose node leaves the cluster.
- **Removal of Crashed Nodes:** the Split Brain Resolver removes a node that has been unreachable for a while, so that the singleton can be recreated elsewhere.

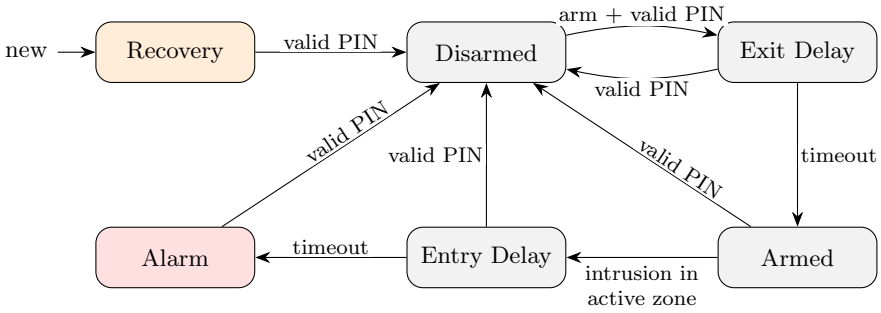



Figure 1: States of the control unit. Every new instance starts in Recovery. Sensor events in Recovery, Disarmed, Exit Delay, Entry Delay and Alarm are only logged; wrong PINs never cause a transition.

# 3 Implementation Overview

## 3.1 Components

- **SmartHomeProtocol:** the message protocol. Messages to the control unit are `SensorTriggered(sensorId, type, zone)`, `ArmSystemRequest(zones)`, `ValidPinEntered`, `InvalidPinEntered`, `RegisterSiren`, `GetState` and `SimulateCrash`. The keypad receives `InsertPin` and `ArmWithPin`, sensors receive `Detect`, sirens receive `ActivateSiren` and `DeactivateSiren`.

- **ControlUnit:** the state machine of Figure 1. It stores the active zones (an empty set means full arming) and the registered sirens. `GetState` replies with the current state, and `SimulateCrash` throws an exception so that the supervisor restarts the actor; both are used for demonstrations and tests.
- **KeyPad:** validates the PIN. A correct PIN becomes `ValidPinEntered` or, for an arming request, `ArmSystemRequest`; a wrong PIN becomes `InvalidPinEntered`. The system is therefore never armed without the PIN.
- **Sensor:** has a `sensorId`, a type (`MOTION`, `DOOR` or `WINDOW`) and a zone. When its hardware detects something it sends `SensorTriggered` to the control unit. Since the event carries the zone, the control unit needs no table mapping sensors to zones, and new sensors can be added without changing it.
- **Siren:** registers itself periodically and switches on and off on request.
- **Main:** starts a node with a list of roles (`control`, `sensors`, `keypad`, `siren`) and a port, creates the actors of those roles, and offers a small console (`trigger`, `arm`, `pin`, `status`, `crash`, `quit`) that only sends messages to the local actors.
- **application.conf:** cluster provider, seed nodes, Split Brain Resolver, serialization binding, PIN and delays.

### 3.2 Deployment

Figure 2 shows the reference deployment on three nodes. Any other split of the roles works in the same way; for example, starting a second `control` node allows the control unit to survive the loss of the first one.

## 4 Testing

### 4.1 Technologies Used

- **Pekko TestKit (`javadsl.TestKit`):** test probes act as sirens and as receivers of state replies, and are used to assert which messages arrive and when.
- **JUnit 5:** structures the test suites (`@BeforeAll`, `@BeforeEach`, `@Test`, `@DisplayName`).

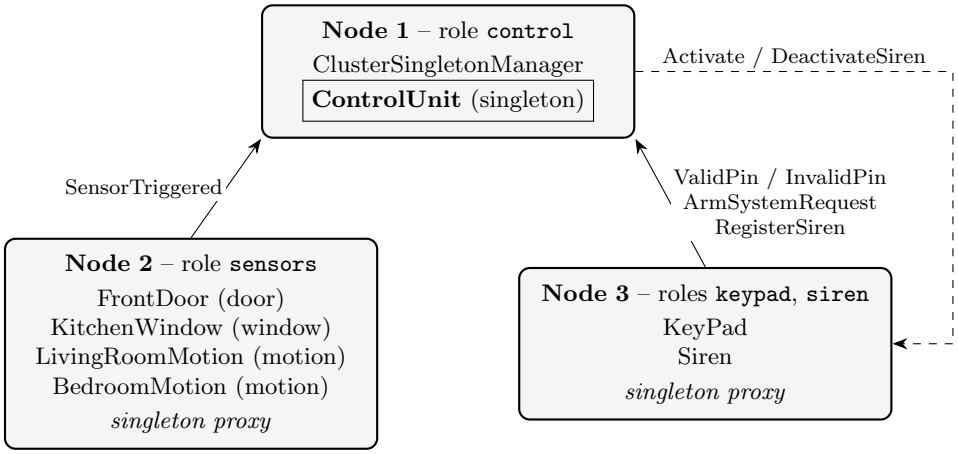



Figure 2: Reference deployment on three nodes. Local actors send to the local proxy, which forwards to the ControlUnit singleton; the ControlUnit answers the siren directly.

### 4.2 Multi-Node Test Setup

The tests run *real* cluster nodes: several `ActorSystem`s inside the test JVM, each listening on its own port on localhost. Every message between a sensor, the keypad, the control unit and the siren therefore goes through remoting and Jackson serialization, exactly as in a deployment on separate machines.

The helper class `TestCluster` provides:

- `startNode(roles)`: creates a node with the given roles, short delays (2 s) and no seed nodes;
- `formCluster(nodes)`: the first node starts the cluster, so it is the oldest one, and the others join it; the method returns when all nodes are *Up*;
- `awaitState(state)`: queries the control unit with `GetState` until it reports the expected state. Tests wait for real state changes instead of fixed sleeps, which makes them independent of the speed of the machine.

Before each test, the probe registers as a siren and the correct PIN brings the control unit to Disarmed (a correct PIN leads to Disarmed from every state), so every test starts from the same state.

### 4.3 Test Scenarios

`SmartHomeClusterTest` (three nodes: `control`, `sensors`, `keypad`):

- **armingWithWrongPinIsRejected:** a wrong PIN does not arm the system.
- **armingGoesThroughExitDelay:** the correct PIN leads to Exit Delay and then, after the timeout, to Armed.
- **sensorsIgnoredDuringExitDelay** and **sensorsIgnoredWhenDisarmed:** sensor events are only logged.
- **intrusionTriggersAlarmAfterEntryDelay:** an intrusion starts the Entry Delay; the siren is *not* activated before the timeout and *is* activated after it; a wrong PIN does not stop the alarm, the correct one deactivates the siren and disarms the system.
- **validPinDuringEntryDelayDisarms:** the correct PIN in time prevents the alarm.
- **partialArmingIgnoresInactiveZones:** in night mode the bedroom sensor is ignored, while a window on the perimeter starts the Entry Delay.
- **validPinDisarmsFromArmed** and **validPinDuringExitDelayCancelsArming:** the correct PIN disarms the system from Armed and cancels an arming in progress.
- **restartedControlUnitEntersRecoveryMode:** while Armed, the control unit is made to crash and is restarted by its supervisor. It enters Recovery; an intrusion does not activate the siren (it does not assume Armed); an arming request is refused (it does not assume Disarmed); a wrong PIN has no effect; the correct PIN switches the siren off and leads to Disarmed, after which the system works normally.

`ControlUnitFailoverTest` (nodes A: `control`; B: `control`, `sensors`; C: `keypad`):

- **recreatedControlUnitStartsInRecoveryMode:** with the system Armed, node A, which hosts the control unit, leaves the cluster. The singleton is recreated on node B, starts in Recovery, ignores an intrusion, and returns to Disarmed only after the correct PIN.

## 5 Conclusion

The actor model handles the concurrency of the alarm system without locks: each component owns its state, the control unit is a state machine driven by messages, and timeouts are messages too. The cluster version keeps exactly the same alarm logic and adds three ingredients: a Cluster Singleton that

guarantees a single control unit and recreates it after failures, a proxy on every node that makes its location transparent, and a recovery mode that every new instance enters by construction, so that the loss of the previous state never leads the system to assume it is armed or disarmed.

**Known limitations.** After a control unit is recreated, a siren may need up to 3 seconds to register again; until then, a new alarm could not be signalled to it (an alarm is not possible in that window anyway, since the new instance is in Recovery until the PIN is entered). If a node crashes rather than leaving gracefully, the singleton is recreated only after the Split Brain Resolver has removed the node (7 seconds of unreachability in the provided configuration). The PIN is stored in the configuration file in clear text, which is acceptable for this exercise but not for a real product.